Publication and Autopaper: From a Feed Item to the Front Page

Doc 5 of the Choir teaching series, written to teach as well as to describe. 14 October 2026. Repository: `origin/main` at 980ffaf4e. Evidence is read from code at that commit and from Choir's own documents; there was no live host access for this series, and nothing on staging or Node B was contacted. Numbers come from the cited problem records and reports, or from constants in the code.

Mutation class of this document: **green** (docs only). Most changes it discusses are **orange** (the front-page rank, prompts) or **red** (Texture canonical writes, the publication path, gateway credentials, restart behaviour).

How to read this

Autopaper is the clearest place in Choir to watch a whole distributed system work: a timer fires, a feed is fetched, bytes are stored by hash, a record wakes a desk, a model labels, a desk decides, another desk writes, a proxy reads back, a host service signs, and a browser ranks. Every one of those hops has failed at least once in the last week, and each failure teaches a general lesson.

Section 1 draws the whole pipeline. Sections 2 to 9 walk it in order: sources and schedules, captures, arrival and intake, stories, Texture writes, quote checking, publication, and the front page. Each section follows the same arc:

1. a **primer** on the general idea, from first principles;
2. **how Choir does it**, with file paths and function names;
3. **where it is weak**, with the incident that showed it;
4. **options and trade-offs**.

Section 10 collects the incidents into one table of lessons. The glossary, five self-check questions and the three files to read come last.

Evidence labels used below: **code** (read in the repository, with a path), **doc** (an existing Choir document), **upstream** (a public standard or vendor document) and **inference** (reasoning that has not been tested; treat it as a hypothesis).

A note on names. Autopaper was once called "World Wire" and "Universal Wire", and its ingestion daemon was "sourcecycled". Those names are retired (CLAUDE.md), but the code rename is a TODO (autopaper SW design §9). So you will still read `UniversalWirePlatformOwnerID = "universal-wire-platform"` in `internal/vmctl/platform_computer.go`, the user agent `ChoirUniversalWire/0.1` in `internal/sources/registry.json`, and the desk verb `choir.SearchWireCorpus` in `internal/yaegikernel/choir.go`. Read every "wire" identifier as "Autopaper". The persisted identity constants cannot be renamed in place without a migration, because vmctl stores them as ownership on the host.

The short answer

1. **Autopaper is a pipeline of single-purpose authorities, not one program.** A schedule polls; corpusd stores captures; management decides what each capture means; Texture alone writes a story; the proxy and corpusd publish; a host timer scores importance; the browser ranks. Each hop has one writer for its facts.
2. **A schedule is a standing intent; a poll is an occurrence.** A crash closes the poll in flight and never resumes it, but the schedule survives and its next tick is new work (`recoverSourceSchedulesAtBoot`, `internal/agentcore/source_schedule.go`). This is how "the paper runs 24/7" coexists with "a crash never resumes work".

3. **Captures are content-addressed and shared.** Each fetched item becomes a `choir.web_capture` object in Store B behind corpusd, with its bytes on host disk under their sha256. Its identity is its URL plus the hash of its normalized text, so a re-poll of an unchanged page is a no-op and a changed page is a new capture beside the old one.
4. **Delivery is at-least-once, so every consumer is idempotent.** Arrival records, story openings and publications all derive their ids from their inputs, so a retry replays instead of duplicating.
5. **Acknowledge after you act, not when you read.** The worst intake bug of the week was a management turn that marked 28 captures read, then failed, so nobody ever handled them (fixed in e3437f1cd).
6. **Story boundaries are a precision and recall problem.** The paper's first measured merge recall, 0.174, was mostly a measurement bug (the scorer saw only 50 stories); re-read, it was 0.630. Measure the measurement before you tune the system.
7. **Quotes are checked against the source's own words, and anchored by hash and by code-point offsets.** A citation that cannot be matched in a captured block is refused at write time. A published version pins the exact block text it cites and refuses a block whose hash moved.
8. **Publication is immutable, hash-chained and signed.** Version n of a story is Texture revision n; each manifest names the previous one's hash and is signed with the platform's Ed25519 key. The guest names only a document and a revision; the proxy reads the revision back from the guest itself before corpusd publishes it.
9. **Reader-facing text needs a reader-shaped gate.** Management's work briefs ("Write the story of ...") became published titles because one string did two jobs. The fix makes Texture's own headline the title and holds a story revision that has none.
10. **The front page ranks importance of the event times depth of our article times recency.** A one-source stub may not lead. Importance is a Jev judgement from title and dek, which is cheap and stable but sees little of the story.

1. The pipeline at a glance

```
source schedule (durable wake, default 15 min, per-source mailbox)
   |  HandleSourcePollDue: fetch feed, no model call
   v
captures -> Store B via corpusd (POST /internal/platform/captures)
   |        bodies on host disk by sha256; object row in corpus Dolt
   |  [optional] Jev labels each new capture: same event as which story?
   v
arrival record (system Report) -> management:<owner> inbox
   |  management (parent or one of K tasks): open, feed or decline
   v
story = Texture document + trajectory (kind story, publish auto)
   |  Texture desk, woken by records, may Cast research
   |  research: corpus first, then web search, Read pages -> captures
   v
Texture revision (head-pinned commit; citations quote-checked)
   |  AutoPublishRevision gates: story? cites a source? leads with a headline?
   v
guest -> proxy /internal/platform/texture/publications
   |  proxy reads doc + revision back from the guest, checks owner
   v
corpusd PublishTexture -> Store B: publication, version n, manifest,
   |  resolved blocks pinned, manifest hash chain, Ed25519 signature
   v
/api/platform/publications/front (Store B only, never the paper computer)
   |  + importance, promo, section from the host scorer (every 10 min)
   v
browser: rankFrontStories = importance x depth x section lift x recency
```

Three properties of this picture matter more than any box in it.

- **Two stores, two lifetimes.** The guest's embedded Dolt holds the paper's working state: inboxes, labels, story documents and their revisions. Store B (the corpus Dolt behind corpusd, on the host) holds the shared resource: every capture and every published version (computer ontology, "Dolt Store Taxonomy"). Doc 1 teaches the stores themselves.

- **Reads never touch the writer.** The public front and the reader resolve from Store B alone (`internal/platform/publication_front.go`, package comment: "nothing here calls a computer, so the front page keeps working while the paper computer is stopped"). The old Universal Wire read every page through the live guest, and every deploy closed the publication window (SW design §0, cause C6).
- **Model calls sit only where judgement is needed.** Polling, capture writing, arrival, publication and ranking arithmetic call no model. Jev (the typed judgement model, doc 3) labels captures, judges citation support and scores importance; the desks make the editorial decisions.

2. Sources and schedules: standing intents and occurrences

2.1 Primer: polling, conditional requests and durable timers

Most news arrives by **polling**: you ask a server for its feed every few minutes. Three ideas make polling cheap and safe.

- **Conditional GET.** HTTP lets a client say "send the feed only if it changed since the version I have." The server gave you an `ETag` (an opaque version tag) and a `Last-Modified` date with the last response. You send them back as `If-None-Match` and `If-Modified-Since`, and an unchanged feed answers `304 Not Modified` with no body (upstream: RFC 9110 §13). The tags are called **validators**. Their trap: if you store a validator before you have safely processed the response, a crash makes the next poll return 304, and the items you never processed are gone.
- **Durable timers.** A timer in process memory dies with the process. A timer that must survive restarts is a row in a database: "wake X at time T with payload P." A scheduler fires due rows. Each firing is an **occurrence** of a standing **intent** ("poll this source every 15 minutes"). The intent and the occurrence have different lifetimes, and confusing them is how systems either stop silently or loop forever after a crash.
- **Exponential backoff.** When a poll fails, wait longer before the next try (double the interval per failure, up to a cap), so a dead feed costs little and a flaky one recovers.

A fourth idea, **generation numbers**, prevents stale timers from acting. Each time an intent is restarted or changed, its generation increments, and every pending timer carries the generation it was armed under. A timer that fires with an old generation does nothing. This is the same trick as a **fencing token** in a distributed lock: the newest holder's number wins, and anything older is refused at the point of action.

2.2 How Choir does it

The registry. `internal/sources/registry.json` lists 220 sources: 146 RSS feeds, 73 Telegram public previews and one GDELT stream (counted from the file's `type` field). Each entry is a `sources.Source` (`internal/sources/types.go`) with polling policy fields (interval, `max_items_per_poll`, `conditional_request_mode`) and rights fields (`tos_class`, `robots_policy`, `store_body_policy`, `retention_days`). One entry has `tos_class: test_fixture`; `StartSourceScheduleRun` refuses to schedule it unless the request is marked as a fixture run, so test data never becomes a standing schedule by accident (L2a R2-7).

The schedule. `StartSourceScheduleRun` (`internal/agentcore/source_schedule.go`) writes a schedule row with the interval (default `defaultSourcePollInterval = 15 * time.Minute`, floor one minute), bumps its `Generation`, sets `NextDue` to now, and calls `armSourcePoll`. That arms a durable actor wake of kind `source_poll_due` on the mailbox `source:<id>` (`sourcePollMailbox`). One mailbox per source means one source's polls run one at a time and a slow feed never holds a desk's mailbox. The wake's payload carries the source id, the generation and the due time.

One poll. `HandleSourcePollDue` does, in order:

1. Under `sourceScheduleMu`, reload the schedule and check it is still active, of the same generation, and due at the payload's time. A stale or replayed wake does nothing, or re-arms the saved next tick.
2. Mark the poll **in flight** durably (`PollInFlightSince`). A crash boot that finds this mark knows a poll was cut.

3. Release the lock and call `researchtools.PollSourceOnce` (`internal/researchtools/poll_source.go`). No model call happens here.
4. If the computer's `intake.labels` setting is on, label the new captures with Jev (section 4).
5. Announce the new captures to management (`announceCaptures`, section 4).
6. Re-take the lock, record the poll's usage on the schedule, clear the in-flight mark, set `NextDue = now + sourcePollInterval(sched)` and arm the next wake.

`sourcePollInterval` doubles the interval per consecutive failure, capped at `maxSourcePollBackoff = 24 * time.Hour`.

Validators and the per-poll limit. `PollSourceOnce` parses up to a hard cap of 100 items (`maxPollMaxItems`), drops items already handled (keyed by item id plus content hash), and writes new ones in batches until it reaches the schedule's limit. Items it did not reach are counted as `Deferred`. The feed's validators are stored only when nothing was deferred and nothing failed (`keepValidators`), so a 304 can never hide an unprocessed item. This ordering was fixed in panel round 2 of L2a: the first design truncated before the repeat check and would have re-read the same first N items forever (L2a design R2-1; intake defects, defect 1). Note that `pollState`, which holds the validators and the handled set, is process memory, not durable state (code). After a restart the first poll simply fetches the full feed and the repeat check in the capture store absorbs the duplicates (inference from the code path: the capture writer looks up each capture id before writing).

Restarts. `recoverSourceSchedulesAtBoot` runs at boot and asks `BootWasPlannedRestart()`:

boot kind	what happens to an active schedule
planned (a marker from the host, consumed once)	keeps its generation and saved tick; the tick is re-armed, which collapses into any wake still pending; a cut poll is redelivered and marks itself again
crash (no marker, or a marker refused)	the in-flight poll is closed as `interrupted_by_restart`; the schedule takes a new generation, so the pre-boot wake is stale; the next tick is armed one interval after boot, never at once
crash, with a poll in flight at 2 crash boots in a row	the schedule is closed with `sourceScheduleCrashLoopWhy`; an owner act restarts it

A crash also closes every schedule's pending-arrival list with the fate "interrupted by a restart". `source_poll_due` is listed in `RestartWorkKinds` (`internal/agentcore/planned_restart.go`), so the generic crash-boot path never re-dispatches the pre-boot wake. The schedule recovery arms a fresh one. Doc 2 teaches the restart rule in general; this is its cleanest worked example.

2.3 Where it is weak

An epoch cutover read as a crash and stopped the paper for an hour. On 14 October at 05:48 UTC a deploy refreshed the live paper. The host wrote a planned-restart marker, and the guest consumed it, but the marker carried work-state epoch 1 while the new binary reads epoch 2 (`CurrentWorkStateEpoch = 2`). `ConsumePlannedRestartMarker` treats an epoch mismatch as a crash on purpose, so nothing resumes into a new schema. Under the code of that morning a crash closed every schedule, so all 14 closed and nothing was published from about 05:48 to 06:49, when the owner restarted them by hand (planned refresh closes paper schedules). vmctl logged `planned_restart=true` because its request succeeded; only the guest console showed the guest's verdict. Two lessons: a rule that is correct for work (do not resume across a schema change) was wrong for standing intents, and the host had no receipt of the guest's own boot classification. The fix (ce59d406) is the intent/occurrence split you see in the table above.

An always-busy computer never takes an update. Deploys never restart a busy computer (CLAUDE.md). With 14 sources polling every 15 minutes the paper is never idle, so it stayed on its old build after the fix landed (same problem record, "After the fix deployed"). Options named there: a short update window, the paper split across several computers that update in turn, or a fast snapshot restart (doc 4).

A pressure hibernate is a markerless boot. The paper was hibernated under memory pressure at 21:00 on 13 October and woke at 23:51 as a crash boot; it was silent for about three hours (same record). The schedule fix keeps the schedules; it does not make the paper exempt from reclaim.

A capture can leave the feed before it is read. An item beyond the per-poll limit is counted as deferred, but if the feed drops it before the next poll, it is gone. Only a per-source cursor would fix that (L2a design R2-1, residual).

2.4 Options and trade-offs

question	option	gain	cost
how a crash treats a schedule	close it (old rule)	no loop risk	the paper stops at every crash-classified boot
	keep it, close only the poll, bound by crash count (current)	24/7 publishing	a poll that crashes the computer runs twice before it is stopped
how a busy paper updates	update window	simple	a few minutes of silence per deploy
	several paper computers updating in turn	no silence	multi-computer coordination, story ownership
	snapshot restart	fast	snapshot path must be trusted (S3 design)
deferred items	larger `max_items`	fewer losses	more captures per poll, more intake load
	per-source cursor	no loss within the source's history	per-feed logic; many feeds have no stable cursor

3. Captures: content-addressed evidence in a shared store

3.1 Primer: content addressing and ordered writes

Content addressing names a piece of data by a hash of its bytes: `sha256:<hex>`. Three properties follow. The same bytes always get the same name, so a second write of the same content is a no-op (**idempotent**). A reader can verify what it got by hashing it again, so a corrupt or substituted file is detected. And a reference to a hash is a reference to exactly those bytes forever, which is what a citation needs.

Identity is a design choice. Hashing raw bytes makes every trivial change (an ad, a timestamp, a nonce in the HTML) a new object. Hashing a **normalized** form (the text, with whitespace and markup stripped) makes cosmetic changes repeats and real edits new versions. The cost is that the normalization rules become part of your identity scheme: change them and every id changes.

Ordered writes. If an object row names a blob, write the blob first, make it durable, then write the row. If you crash between the two, you have an orphan blob (harmless garbage) instead of a row that names missing bytes (a broken reference). The standard way to make a file appear atomically is: write a temporary file in the same directory, `fsync` it, then `rename` it to its final name. On POSIX filesystems a rename within one directory is atomic, so a reader sees either no file or the whole file.

Who may write. A shared store needs an authorization rule per write: which caller may write which rows. Choir binds a guest's request to its computer by the **source address of its tap interface**: the host's vmctl knows which guest owns which address. This is network-address identity. It is cheap and unforgeable from inside a correctly firewalled guest, and it breaks as soon as addresses move or a non-guest process shares the network (see the security review for the loopback side of this).

3.2 How Choir does it

The write path. A guest's `PollSourceOnce` calls `deps.Captures`, which posts the items to corpusd's `POST /internal/platform/captures` (`HandleCaptures`, `internal/platform/captures.go`). A guest reaches corpusd directly at its platform URL, not through the proxy (a correction recorded in capture route unreachable through proxy). corpusd checks the caller with `requireInternalOrBoundGuestWrite` (`internal/platform/authority.go`): either a trusted host-internal caller (loopback plus `X-Internal-Caller: true`), or a guest whose tap address vmctl binds to exactly the owner and computer the request names. It bounds the request (100 items, 16 MiB raw bytes per item, 64 MiB per request, `maxCapture*` constants) and calls `sourcegraph.WriteCaptures`.

One capture (`WriteCaptures`, `internal/sourcegraph/captures.go`):

1. Split the item into **blocks** (paragraph-sized pieces with content ids, `sourcefetch.TextBlocks` for a feed item, or the reader's blocks for a page that `choir.Read` fetched).
2. Compute `docHash = sourcefetch.DocHash(blocks)` and the identity key `"autopaper:capture:" + canonicalURL + "#" + docHash`; build the canonical id from it.
3. If an object with that id exists, count a **repeat** and stop.
4. Store three blobs through the body store: the raw bytes, the block JSON (`choir.capture_blocks.v1`) and the text projection.
5. Write the `choir.web_capture` object with `CreateWebCaptureIfAbsent`. If another writer created it in between, count a repeat; its capture stands.

The body store (`CaptureBodyStore.PutBody`) writes `<root>/<first two hex chars>/<hash>` with the temp-file, fsync, rename pattern, skips an existing hash, and `Body` re-hashes on every read and refuses bytes that fail their hash. Bodies live under the platform artifacts root (`CaptureBodiesRoot`), never in Dolt rows: that is the lesson of the old 104 GB store, where unbounded ingest into a versioned database made history unmanageable (SW design §2).

Reading. Any bound guest may read any capture body (`HandleCaptureBody`, `allowAnyGuest = true`), because captures are shared content. Desks read them through `choir.ReadCapture`, which pages blocks.

3.3 Where it is weak

The first build put captures on the wrong computer. Station W1b wrote captures into the polling computer's own embedded graph. The owner: the paper "shouldnt run on embedded dolt, but global dolt ... the point is to make it a resource for everyone" (captures built per computer). The store taxonomy already said so; the design doc of 10-09 had drifted from it, and the station was written from the design doc without checking the taxonomy. This is standing question 2 (settled-decision conformance) and its July receipt repeating itself. The general lesson: before you author a mission, diff its design against the settled decisions it inherits.

A cause named from one variable was wrong. The same day, a capture write failed, and the first diagnosis was "guests reach corpusd through the proxy, which does not forward the route." The correction: the guest's `CHOIR_PLATFORM_URL` comes from the kernel command line and takes precedence over `RUNTIME_CORPUSD_URL`; guests reach corpusd directly. The real cause was that the test computer still ran a superseded build (capture route, correction). Lesson from the record itself: "read which setting wins before naming a cause from one variable."

Re-captures inflate everything. Identity is URL plus normalized text. Hacker News feed items carry live point and comment counts in their text, so every poll that sees a changed count writes a new capture. In one six-hour window, 146 of 425 captures (34%) re-captured a URL already captured, 120 of them with an unchanged title (L2a merge recall, finding 4). They cost intake work and create spurious judge pairs.

Retention is designed, not enforced. Capture bodies were to shrink to a stub after a bounded window, with cited bodies pinned; "no code enforces the capture body retention window" (state and vision §3.8).

Network-address identity has a sharp edge. The tap binding is only as good as the address map. The same family of bug disabled Jev on the paper for a whole morning (section 4.3). And loopback counts as trusted: any process on the host that sets `X-Internal-Caller: true` from 127.0.0.1 is an internal caller (`trustedInternalTransport`, code). The isolation review §3.5 treats that as a High finding for every host service, corpusd included.

3.4 Options and trade-offs

question	option	gain	cost
capture identity for churny feeds	URL + full text (current)	any real edit is a new version	counter churn is a new capture
	URL + text with known volatile spans removed (per source rule)	far fewer re-captures	per-feed rules; a rule can hide a real edit

question	option	gain	cost
	URL + title, new version only on body change beyond a threshold	simple	thresholds are arbitrary; misses small corrections
guest write identity	tap source address (current)	no secrets in the guest	breaks if addresses move or the firewall leaks
	per-computer bearer bound to the computer, checked at corpusd	survives address change	a credential to protect inside the guest
	both (address and bearer must agree)	defence in depth	two things to keep consistent
retention	none (current)	simple, every quote re-checkable forever	unbounded disk, rights exposure
	30-day window, cited bodies pinned (designed)	bounded	a pin index; a GC that must never delete a cited body

4. Arrival and intake: labels, dispositions, backpressure

4.1 Primer: delivery guarantees, acknowledgements and queues

A message system can promise one of three things. **At-most-once**: a message may be lost, never duplicated. **At-least-once**: never lost, may be duplicated. **Exactly-once** delivery is impossible in general across a network failure; what systems actually build is at-least-once delivery plus an **idempotent consumer**, which gives exactly-once *effect*. The usual tool is a **deterministic id**: derive the message's id from its content, so a retry produces the same id and the store refuses or replays it.

A consumer that reads a queue keeps a **cursor** (how far it has read). When it moves the cursor is the whole game:

- move it **when you read** (ack-on-receive): fast, but a crash or failure after reading loses the messages;
- move it **after you act** (ack-after-process): a failure leaves the messages unread, so they are retried, and the consumer must tolerate seeing them twice.

Ack-after-process is the default for anything that must not be lost.

Queues and Little's law. For a stable queue, the average number of items in the system L equals the arrival rate λ times the average time each spends there W: L = λW. Its practical corollary: if arrivals exceed the service rate, the queue grows without bound and every item waits longer. There are only three answers: serve faster (more workers, cheaper work per item), admit less (sample, batch, shed), or accept growing latency.

Leases and optimistic concurrency. When several workers share work, each item is **leased** to one worker for a bounded time, so two workers never act on it at once. When workers write to a shared structure (a story library), **optimistic concurrency** lets each one work without locks and checks at commit time that what it read is still current: "commit only if the head is still 41." If the head moved, the commit is refused and the worker re-reads.

4.2 How Choir does it

The arrival record. After a poll writes captures, `announceCaptures` (`internal/agentcore/capture_arrival.go`) appends one system Report record to `management:<owner>`. Its id is `types.CaptureArrivalRecordID(sourceID, ids)`, a pure function of the source and the capture set, and its command digest covers the whole record. So a retry naming the same captures replays the committed record, and `ErrLifecycleCommandConflict` is read as "already announced". If the append fails, the captures go on the schedule's `PendingArrivals` list and the next poll names them first; the list is bounded at `sourceArrivalPendingLimit = 200`, oldest dropped with a count and a log line. A crash boot closes the list ("interrupted by a restart").

Labels. With `intake.labels` on, `labelCaptures` (`internal/agentcore/intake_labels.go`) asks Jev one question per capture: which candidate reports the same real-world event, or `new`. The candidates are the 30 newest stories plus 10 more by title-token overlap within a 72-hour horizon (`intakeShortlistRecent`,

`intakeShortlistOverlap`, `intakeShortlistHorizon`), plus the poll's other new captures as siblings (`n1, n2, ...`). Choices are short aliases mapped back to document ids in code, so the model never handles raw ids. A second question in the same request asks which story shares the **topic**. The last 12 management overrides ride along as worked corrections. Budgets: 4 labels at once, 5 seconds per call, 15 seconds per poll, on the gateway's own `intake_labels` rate scope so labelling cannot starve the citation support judge. A label is its own `choir.capture_label` object, never part of the arrival record: the record's digest covers its content, so a label that differed on retry would make the replay conflict. **Labels never gate arrival**: a timeout or refusal leaves the capture unlabelled.

Dispositions. Management answers each capture with one act: open a story (`choir.Cast("texture:new", headline, spec)`), feed it into one (`choir.Feed(doc, captureIDs, reason)`) or decline it (`choir.Decline(captureIDs, reason)`) (`internal/yaegikernel/choir.go`). The runtime compares each disposition with the label and records an override on a mismatch, which becomes a worked example for the next labels.

Delegation and its guards. Management delegates batches of captures to **management tasks** (S2a, then S2b). `managementTaskSlots` (`internal/agentcore/management_tasks.go`) runs up to K at once (default 4, at most 8; the owner setting `management.serial_tasks` drops it to 1 as a live rollback). Coherence comes from the store, not from K (`internal/store/management_tasks.go`, header comment):

- a **lease**: the task's open round is the lease on its captures;
- a **per-capture disposition index**: one capture, one disposition; a second disposition, or one of a capture another task leases, is refused;
- a **story-library head**: a counter of story openings. A `texture:new` from a run whose last read of the library is behind the head is refused (`planStoryLibrary`, `ErrStoryLibraryMoved`), so two tasks cannot both open a story for one event without one of them first seeing the other's.

That last guard is optimistic concurrency, exactly as in the primer. Doc 2 covers delegation as a desk mechanism; here it is the intake's throughput lever.

4.3 Where it is weak

Read is not done: 28 captures lost to a failed turn. At 08:22 UTC on 14 October, 28 captures arrived in a burst while the model provider's circuit was open. A management run's first cell only *read* them, and the runtime committed a cursor advance for what that cell had shown. The run then failed. Nothing undid the advance and nothing re-showed the records, so the 28 captures were never disposed (captures undisposed after failed management runs). This is ack-on-receive in disguise. The fix (e3437f1cd) moves the read mark to the end of a completed turn, and after a failed turn schedules one durable retry with backoff (2, 4, 8, 16 minutes), bounded so a provider outage cannot become a hot loop. Residual named in the record: a turn that disposes some captures and then fails still marks all of them read; fixing that needs a per-capture disposition index, which S2b's tasks now have but the parent desk path does not (as recorded; not re-verified here).

Jev labelled nothing for a morning. `intake.labels` was on, yet every poll recorded `labelled: 0`. The guest log said "Jev bearer is not authorized for this VM". The gateway pinned each credential to the first guest address that ever called Jev (`BindJevPeer`), but a computer's guest address comes from a host port allocated at each boot, while its credential persists across boots. After any cold boot on a new port, Jev refused that computer (L2a merge recall, finding 2). In the code at this commit, the credential ensure path that vmctl calls for every ready computer now moves the pin to the current address (`JevPeerMoved`, `internal/gateway/config.go`), so only host lifecycle control can move it (code). Lesson: two identity facts with different lifetimes (a token that persists, an address that does not) must be reconciled by the authority that knows both.

Intake fell behind at 28 sources. With one task slot, arrivals ran at about 270 captures an hour while 8 tasks completed in 34 minutes; queue wait was p50 425 s and p95 1,506 s, task duration p50 241 s (management intake backlog). By Little's law the backlog had to grow. With a p50 task time of about four minutes, one slot serves

roughly 15 tasks an hour (arithmetic); the only answers are more slots (S2b's K = 4), cheaper dispositions (Jev settling high-confidence feeds without a model turn, ruling O3), or fewer arrivals (longer intervals on low-yield, high-churn feeds).

The runtime accepted a Cast with an empty capture id. A cell that set capture variables failed to compile; the next cell used them empty and opened 7 stories naming no capture (same record, finding 4). A task's dispositions must now name captures leased to it, which refuses a stray id inside a task (code, `taskPlan.dispose`); whether the parent desk path now refuses an empty or mangled id was not verified for this document.

4.4 Options and trade-offs

question	option	gain	cost
when to mark records read	at read (old)	simple	loses work on failure
	at the end of a completed turn (current)	nothing lost	a failed turn re-shows records; consumers must be idempotent
	per item, as each is disposed	exact	a disposition index on every path
intake throughput	K parallel tasks (S2b)	linear-ish speedup	coordination guards; more spend at once
	Jev settles high-confidence feeds with no model turn (O3)	far cheaper per capture	trusts a cheap judge; needs an override path
	admit less (intervals, `max_items`)	immediate	coverage loss
label failure	never gate arrival (current)	intake survives Jev outages	silent quality loss unless coverage is counted
	gate arrival on a label	every capture labelled	Jev becomes a single point of failure for the paper

The design chose "never gate" and counts `labelled` and `unlabelled` on every poll. That count is what made the morning's outage visible at all.

5. Stories: clustering, merge recall and entity resolution

5.1 Primer: precision, recall and the shape of the problem

Deciding which captures belong to one story is **clustering**, and its cousin in databases is **entity resolution**: deciding which records refer to the same real thing. Both are judged by pairs.

- Take every pair of items that truly belong together (judged by a reference). **Recall** is the share of those pairs that ended in the same cluster. Low recall means duplicates: three stories about one event.
- Take every pair you put together. **Precision** is the share that truly belong together. Low precision means false merges: two events in one story.

You can always buy one with the other. Merge everything and recall is perfect; merge nothing and precision is. The interesting work is moving both.

At scale you cannot compare every pair, so you **block**: build a short candidate list per item (recent stories, shared words, shared entities, embedding neighbours) and compare only within it. A blocker needs high **recall**, because an item whose right cluster is not on the list cannot be merged however smart the final judge is. Precision can come from the judge.

Three hazards recur.

- **Topic versus event.** "Russian strikes cut power in Kyiv" and "Drones shot down off Bulgaria" share actors and a war; they are different events. A judge that merges by topic produces plausible-looking wrong stories.

- **Transitivity.** If A joins B and B joins C, A and C are now together whether or not they belong. Chains of weak links grow giant clusters.
- **Measurement validity.** Your metric is only as good as the code that enumerates the clusters. A scorer that cannot see half the clusters reports low recall for a system that is fine.

5.2 How Choir does it

A **story** is a Texture document with a trajectory of kind `story`; its boundary is the set of captures management opened it from or fed into it (L2a design A3). Opening is deterministic: `storyOpeningIDs` (`internal/store/story_open.go`) derives the document, trajectory and seed revision ids from (owner, computer, command id, record id) with `uuid.NewSHA1`, so a retried commit replays and cannot create a second story. A **topic** is the set of stories joined by Jev's "new, but same topic as" answers; in this slice it is recorded only on the label.

The blocker is the label shortlist from section 4.2 (recency first, then title-token overlap). The judge is Jev, then management. Corrections flow back as overrides that become examples, so boundaries improve from the ledger with no training step.

The owner's measure is not merge recall but **articles per source**: "we should be able to scale up the sources while only scaling up article production significantly sublinearly" (L2a merge recall). The baseline window: 70.8 captures an hour, 30.2 new stories an hour, 2.35 captures per new story, 1.65 distinct pages per new story, and only 7.4% of stories with two or more intake outlets. Read plainly: almost every distinct page became its own story.

The longer-term design moves the merge key from "story" to **event**. A claims, events and entities layer extracts attributed claims from captures, resolves the events they report, and lets "same event" be decided by shared participants and time rather than by headline text (claims, events and entities §9.1). Merges and splits there are recorded **judgments** with a judge and a probability, so they can be reversed.

5.3 Where it is weak

The first score was mostly the scorer. After-window 1 reported merge recall of 0.174 (8 of 46 judge pairs). The window scorer found each capture's story by listing `GET /api/texture/documents`, which is capped at 50 documents, newest first. The window's dispositions named 175 stories. Re-read across every story inbox, recall was 0.630 (29 of 46), and the share of captures attributed to a story rose from 13% to 60% (same record, finding 1). A 50-row cap built for a UI list had quietly become a measurement instrument. The same cap was management's only view of the library until `choir.Stories` replaced it (defect 3).

The real misses had four causes, only one of them editorial. Of the 17 pairs that truly missed: 10 were the failed-turn incident above, 3 were re-captures of a page already in the story, 2 were a second story opened for an event that already had one, and 2 were a feed into a story on the same topic but a different event (same record, finding 3). Of 31 candidate false merges, 10 were wrong, all of them topic-level feeds ("feed to existing Ukraine coverage").

The conjecture was not tested. Because Jev labelled nothing that morning, management routed by keyword-filtering headlines. The window measured management without labels, not the L2a mechanism. The record says so explicitly, and names prompt changes for the topic/event errors as "tuning without the mechanism under test". That restraint is the convergence rule at work: understand before tuning.

5.4 Options and trade-offs

question	option	gain	cost
blocker	recency + title tokens (current)	no embeddings, cheap	weak across languages and for older stories
	+ embedding neighbours (qwen3-embedding is in `internal/qdrant`, unwired)	cross-language recall	an index to run and keep fresh
	+ shared entities and time (event layer)	precise "same event" candidates	the claims layer must exist first

question	option	gain	cost
judge	Jev labels, management decides (current)	cheap advice, editorial control	management's turn cost on every capture
	Jev settles high-confidence feeds alone (O3)	sublinear cost	a cheap judge's mistakes publish
boundary repair	overrides as examples (current)	learns with no training	only as good as override reasons
	explicit split and merge acts on stories	fixes past mistakes	published versions are immutable; repair is a new version

6. Texture writes: one writer, head-pinned revisions

6.1 Primer: single writer and compare-and-swap

Many consistency problems vanish if each fact has **one writer**. Two writers to one document need locks, merges or conflict resolution; one writer needs none. The cost is that every other party must *ask* the writer rather than write, and the writer becomes a throughput and availability bottleneck for that fact.

Even one logical writer can run twice: a retried job, a replayed cell, two turns racing. So the commit itself must be conditional. **Compare-and-swap (CAS)** on a head pointer says: "make revision R the new head, only if the head is still H, the revision I based R on." If someone else moved the head, the commit fails and the writer rebases. Combine CAS with an **idempotency key** (a deterministic id per attempt) and a replay of the same attempt returns the original result instead of failing the CAS.

6.2 How Choir does it

Texture is the sole writer of canonical document revisions (CLAUDE.md, Desk Ontology). In code, a Texture cell builds one `yaegikernel.TextureTurn` with `choir.Write`, `Done`, `Wait` or `Reject`, and the reducer commits it through `CommitCellTextureAuthor` (`internal/textureowner/texture_cell_author.go`), whose header states "nothing else here mints one". Two mechanisms make the commit safe:

- **Idempotent replay.** The reducer derives a deterministic `revisionIdentity` per cell. A replayed cell finds the receipt already recorded on the run tape and returns it instead of committing again.
- **Head pinning.** `ApplyTextureTurn` (`internal/store/texture_turn.go`) requires `ExpectedHeadRevisionID` and refuses the commit if the document's current revision is not that id. The new revision's parent is the expected head.

`choir.Write` takes a script with citation tokens and materializes it (`internal/texturewrite/apply.go`). Each citation must name a **captured** source; a quote must match a block of it (section 7). Otherwise the write is refused with a line-located error that tells the desk what to do: "is not captured: a citation needs a captured copy of the source to check its quote against. Ask research to Read it." That refusal is the mechanism that turns "cite your sources" from a prompt into an invariant.

The title comes from Texture. In the same commit that moves the head, `storyTitleAfterRevision` (`internal/store/story_open.go`, called at `texture_turn.go:438`) sets a story's title to the heading its revision leads with (`texturedoc.Headline`, `internal/texturedoc/headline.go`), cut to 160 runes. A headingless revision keeps the old title, and a document that is not a story never changes title. Title and head are one object write, so they cannot disagree.

6.3 Where it is weak: one string, two jobs

On 14 October, 15 of 100 front-page titles were desk instructions: "Write the story of Switzerland proposing to ban profiling-based advertising aimed at minors: what the Swiss federal government has actually published...", "Open a story on the Mediaite report..." (instruction text published as story titles). The trace:

```
choir.Cast("texture:new", objective, spec)
  -> Directive.Objective            also Texture's work brief
  -> planStoryObjects: Title = storyTitle(objective)   (160 runes)
  -> Texture revisions: head moves; Title never touched (before the fix)
  -> proxy publishTextureRevision: Title = doc.Title
  -> front row title
```

The management prompts already said the objective is the headline. The model sometimes wrote a brief instead, because the same field is also the brief Texture reads. Meanwhile 24 of the newest 50 story revisions led with a real headline that nothing downstream read. The record calls this "a substrate cause, not a prompt slip": the only enforcement was the model obeying, and the field's other job pulled the other way.

The sibling: Texture's V1 of an owner's document is "a fast acknowledgment ... the plan", which is right for a private document and wrong for a story that publishes every revision. "This document will explain the dispute between Kemi Badenoch and Nigel Farage..." became a published dek.

The fix (red class; described in the record and present in code): Texture's headline titles the story, and `AutoPublishRevision` holds a publish-auto story revision with no leading headline, as it holds one that cites no source (`internal/agentcore/story_publish.go`). The prompt (`internal/textureprompts/texture.yaml`) states the rule. Published versions are immutable, so leaked titles are repaired by the story's next headline-led revision, and the route slug fixed by v1 (`/pub/texture/write-the-story-of-...`) stays forever, because links never change.

The general lesson is about **channels**. Work-facing text (instructions, plans, briefs) and reader-facing text (titles, deks) must travel in different fields, and every reader-facing field needs a gate shaped like a reader's expectation. This is the same structure as prompt injection (section 9.1), seen from the other side: there, data is read as instruction; here, instruction was published as data.

6.4 Options and trade-offs

question	option	gain	cost
who titles a story	management's opening objective (old)	available at open	a brief can become a title forever
	Texture's leading heading (current)	the writer titles its own story	headingless revisions are held, so gaps appear until Texture complies
	a separate `headline` argument on the opening Cast	clear channel from the start	management may still write a brief there
V1 of a story	an acknowledgment and plan	fast feedback for owners	publishes as news
	news from the first version (current prompt)	every version is readable	V1 waits for research

7. Quote checking and anchors: bytes, code points and hashes

7.1 Primer: what a "character" is

Text has at least four units, and bugs live in the gaps between them.

- **Bytes.** UTF-8 encodes each character in 1 to 4 bytes. "é" is 2 bytes, "中" is 3, and an emoji such as U+1F600 is 4. Go strings index by byte.
- **Code points.** A Unicode code point is one abstract character number (U+00E9). Go's `rune` is a code point; `utf8.RuneCountInString` counts them. JavaScript strings are UTF-16, where characters above U+FFFF take two units, so `"\u{1F600}".length` is 2; `Array.from(s)` iterates by code point.
- **Grapheme clusters.** What a reader sees as one character can be several code points: "é" written as "e" plus a combining accent, a flag (two regional indicators), or a family emoji joined with zero-width joiners. Cutting inside a cluster renders garbage.

- **Normalized forms.** Unicode has several byte sequences for "the same" text. **NFC** composes ("e" + accent becomes "é"); **NFKC** also folds *compatibility* characters (full-width " A " to "A", the "fi" ligature to "fi", "…" to "..."). Normalization is how you compare text that came through different keyboards, encoders and web pages (upstream: Unicode Standard Annex 15).

An **offset** is meaningless without its unit. A byte offset computed in Go and read as a JavaScript index highlights the wrong words as soon as any non-ASCII character precedes the quote.

Two more tools complete the kit.

- **Offset maps.** If you search in a normalized copy of the text, you must map the match back to the original, because the original is what you show and hash. Keep, for each byte of the normalized text, the source range it came from.
- **Anchors that detect drift.** A citation that says "block X, characters 10 to 42" goes stale if the block changes. Store a hash of the block's text (or of the covered words) beside the offsets, and refuse or fall back when the hash no longer matches. The W3C Web Annotation model calls these a *text position selector* and a *text quote selector*; robust systems carry both (upstream: W3C Web Annotation Data Model, "Selectors").

7.2 How Choir does it

One normalization for addressing and matching. `internal/normtext` defines three levels (`normtext.go`, package comment):

level	rules	used by
`LevelCanonical` (`Normalize`)	NFKC; invisible format characters dropped; curly quotes, apostrophes and dashes folded to ASCII; whitespace runs collapsed to one space; trimmed; case kept	block content ids
`LevelFolded` (`Fold`)	canonical, lowercased	the matcher's whitespace-and-case tier
`LevelLoose` (`Loose`)	folded, with every rune that is not a letter, digit or space dropped	the matcher's punctuation tier

`Map` returns the normalized text with a `Start` and `End` source byte offset for every output byte, always covering whole source runes, so a match in normalized text maps back to whole characters of the original.

A block's **content id** is `CID(kind, text)`: the first 16 hex characters of `sha256(kind + "\x00" + Normalize(text))`. Whitespace changes keep the cid; any change of words, case or punctuation changes it. Duplicate blocks in one document get `cid.1`, `cid.2` (`AssignCIDs`). Because the cid and the matcher share one normalization, a block's address and a quote's match cannot disagree about the same words. `normtext.Version = "nt1"` names the rules; changing them re-addresses every cid, so it must bump the version.

The matcher. `quotematch.Match(quote, blocks)` (`internal/quotematch/quotematch.go`) ports a cascade from an earlier project: exact bytes, then whitespace-and-case tolerant, then punctuation tolerant; the first tier that matches wins, and an exact match in a later block beats a tolerant one in an earlier block. Two rules are stricter than its source: a quote must lie within **one block**, and ellipsis segments ("..." or U+2026) must appear **in order** in that block. The rendered quote is always the source's own words (`Result.Excerpt`, the matched spans joined by " ... "), never the quote as the model wrote it. A tolerant match is reported with its tier, so the reader can mark it as approximate.

From match to anchor. In `writer.citation` (`internal/texturewrite/apply.go`), a matched quote fills a `types.Ref` (`internal/types/ref.go`): the capture (`Doc`), its version (`docHash`), the block cid, the block's hash, the excerpt, and since fdaca19de (H-OG3) a `Span`. `RefSpanOf` converts the matcher's byte offsets to **code points** of the block's original text, one range per ellipsis segment, and records `TextHash`, the sha256 of the covered bytes. It returns nil if any offset falls inside a multi-byte character, so a Ref never carries a span that points between bytes.

At publish. `resolveAnchors` (`internal/platform/publication_versions.go`) re-reads each cited capture from Store B, refuses a Ref whose `Version` is not the capture's current `docHash`, refuses one whose `Hash` differs from the block's text hash, and pins a copy of the block's text into the version (`ResolvedBlock`). A published version therefore carries its own evidence: the reader shows the pinned block, not a pointer to a capture that might later shrink to a stub.

In the reader. `splitAtSpan` (`frontend/src/lib/texture-published-citations.ts`) cuts the pinned block with `Array.from(text)`, which counts code points exactly as Go's rune count does, marks each segment, and accepts the result only if the marked words joined by " … " equal the quote. Otherwise it falls back to `splitAtQuote`, a substring search.

7.3 Where it is weak

The span was computed and thrown away for weeks. Until fdaca19de, the matcher returned spans and the writer kept only the block and the excerpt. The reader re-found the quote with `indexOf`, so a phrase that occurred twice in a block was marked at the first occurrence, not the one verified, and an ellipsis quote was never a substring and rendered unmarked (object-graph gaps, H-OG3). The general lesson: when a verifier computes *where*, persist the where; re-deriving it later is a second, weaker verifier.

Code points are not graphemes. Spans count code points on both sides, which is consistent. But a span edge can in principle fall inside a grapheme cluster (between a base letter and its combining mark, or inside a joined emoji), and `Array.from` would render half a cluster (*inference*: the matcher's segment boundaries come from normalized-text matches mapped to whole source runes, not whole clusters; no test for this case was found).

The reader checks words, not the hash. `splitAtSpan` compares the marked words with the quote string; it does not recompute `TextHash`. Since the block text is itself pinned and hash-checked at publish, this is probably sufficient (*inference*), but the hash is carried and unused by this reader.

NFKC is generous. Folding compatibility characters means the tolerant tiers accept a quote that differs from the source in full-width forms, ligatures or superscripts. That is usually what you want for quotes typed by a model; it also means "tolerant" matches should be shown as such, which the tier badge does.

Normalization is part of identity. A `golang.org/x/text` upgrade that changes NFKC tables for characters in captured text re-addresses cids. The package comment says this is a Version bump; no test pins the tables (*inference* from the file listing: `normtext_test.go` tests behaviour, and I did not find a table-hash test).

7.4 Options and trade-offs

question	option	gain	cost
offset unit	bytes	native in Go	wrong in every browser for non-ASCII
	code points (current)	same count in Go and JS (`Array.from`)	can cut a grapheme cluster
	grapheme clusters	what readers see	needs a segmentation library on both sides, with matching versions
matching strictness	exact only	nothing approximate is published	refuses quotes with curly quotes or spacing differences
	tolerant cascade with tier shown (current)	accepts honest variants, labels them	a tolerant match can join words the source separated by punctuation
anchor robustness	position only	simple	silent drift
	position + quote + block hash + span hash (current)	drift detected, fallback possible	more fields to carry through every path

8. Publication into the Autopaper object graph

8.1 Primer: immutability, hash chains, signatures and deputies

Immutable versions. A published version that can change is not a citation target; nobody can point at "what the story said at 14:32." So publication systems make each version immutable and give it a permanent link, and a "latest" link that moves.

Hash chains. If each version's manifest includes the hash of the previous manifest, then changing any old version changes every later hash. A reader holding the latest hash can detect tampering with history. Git commits and certificate-transparency logs use the same idea.

Signatures. A digital signature (Ed25519 here) over a manifest hash proves that the holder of a private key approved exactly those bytes. It proves nothing about whether the content is true, and it is only as trustworthy as your knowledge of *which* public key is the publisher's. A signature that carries its own public key inline proves consistency, not origin, unless the verifier pins that key from somewhere else.

The confused deputy. A deputy is a privileged service acting on behalf of a less privileged caller. It is confused when the caller tricks it into using its privilege on something the caller could not touch: "publish this document" where the document is someone else's, or where the caller hands over forged content. The defence is to let the caller only *name* things, and have the deputy fetch them itself from the authority, checking ownership as it goes.

Idempotent publish. Publishing the same revision twice must return the same version, not mint a second one; publishing an older revision after a newer one must not reorder history.

8.2 How Choir does it

In the guest: gates and an ordered queue. After a Texture revision commits, `AutoPublishRevision` (`internal/agentcore/story_publish.go`) publishes it only if:

1. the document has a trajectory of kind story opened with publish auto (`types.StoryPublishesAuto`);
2. the revision is not the story's seed;
3. it cites at least one captured source (`texturewrite.CitesSource`);
4. it leads with a headline (`texturedoc.Headline`).

A revision that fails 3 or 4 is **held**: it stays committed, its version number becomes a gap, the reason is logged and returned to the turn. A passing revision is queued on an in-process queue keyed by story (`enqueueStoryPublish`), so v is attempted before v<n+1>. The queue is deliberately not durable: a crash loses it, no boot step refills it, and an unpublished revision stays a gap. The next revision publishes as its own number. This is the restart rule applied to publication (package comment, "no boot-time resumer").

In the proxy: name, do not send. The guest posts only `{doc_id, revision_id}` to `/internal/platform/texture/publications` (`types.StoryPublishPath`). `HandleGuestTexturePublication` (`internal/proxy/platform_publish.go`) binds the caller to a live computer (`boundProxyGuest`), then `publishTextureRevision` **reads the document and the revision back from that computer**, checks that both belong to the bound owner, checks that source entities have matching `source_ref` nodes, and builds the publish request itself. `publish_mode` is set by the route (`auto` for a computer's own publish), never by the request body. The guest cannot hand the proxy forged content, and cannot publish another owner's document.

In corpusd: one publication, version n. `Service.PublishTexture` (`internal/platform/service.go`) runs under a process-wide `writeMu` and:

- derives the publication's identity from (owner, source doc): `"pub-" + sha256("texture-publication\x00" + owner + "\x00" + doc)[:32]` (`publicationIdentityFor`, `publication_versions.go`), so every revision of one story lands in one publication;

- returns the existing version unchanged if this revision number was already published (`replayedPublication`), and refuses a revision below the latest published one ("a version that failed to publish stays a gap");
- fixes the route at v1: the slug, publish mode and owner of the publication never change, "whatever later titles or slugs say";
- pins every cited block (`resolveAnchors`, section 7.2);
- writes a manifest (`choir.platform.publication_version_manifest.v1`) holding the content hash, the source revision hash, the anchors, the resolved-blocks hash and the **previous manifest's hash**, so the versions form a hash chain;
- signs the manifest hash with the platform's Ed25519 key (`signManifest`, `internal/platform/signing.go`) and stores the signature with its key id and public key.

The permalink of version n is `<route>/v<n>`; the bare route serves the latest (`versionPermalink`, `parsePublicationRoute`). Citation support verdicts from Jev travel separately to Store B (`publishStoryVerdicts`) and appear as a live **check overlay** beside the immutable payload, so a verdict can arrive after publication without changing the version.

The write gate. Every durable object-graph write, on both the guest's Dolt store and Store B, now passes `objectgraph.CheckWrite` (`internal/objectgraph/registry.go`) first, which refuses an object or edge whose kind is not registered (7f287a135, H-OG1). Before that commit the registry was consulted on one write path only, and 26 object kinds and 12 edge kinds were being written with no registration (object-graph gaps). Replay and migrations stay ungated, because replay must never refuse history. The dead kind `choir.universal_wire_story_cluster` was deleted (427d14ffd, H-OG2); its string stays in the frozen decode table, because old computers may hold such rows.

Adoption: designed, not built. The Autopaper object graph is meant to be adopted: another computer references public objects onto its own tape by id and content hash, judges them as it likes, and keeps its judgments private until it publishes them (claims, events and entities §9.7). Today a computer can read shared captures from Store B; the overlay and adoption records are design (no adoption code for Autopaper objects was found in this commit).

One name trap: `internal/receiptsigner` is a different signer. It signs guest **receipts** (for example `ExecutionIdentity`) for the computer's event chain, over a unix socket, with a per-computer key. Publication manifests are signed by corpusd's own platform key in `internal/platform/signing.go`.

8.3 Where it is weak

Gaps are permanent by design. A transient corpusd error during publish loses that version forever; only a later revision publishes. For an active story this is invisible; for a story that receives no further evidence, the last revision may never reach readers (*inference* from `AutoPublishRevision`: there is no retry and no boot sweep). The trade is deliberate (no resumer that could loop), and a bounded, logged retry of the latest unpublished revision would keep it while reducing loss.

The signing key lives where root processes can read it. The platform signing key is a file on Node B. The isolation review found that host services and every Firecracker VMM run as root (isolation review, short answer 1 and 3). A VMM escape could sign manifests as the platform.

A self-carried public key proves little. Each signature stores its own public key. A verifier who trusts that inline key accepts any key; the platform's key must be published somewhere independent (a well-known URL, DNS, a transparency log) for signatures to prove origin (*inference*: no key-publication endpoint was looked for in this review).

One mutex serializes all publishes. `writeMu` is held across a whole publish, including reading and hashing every cited capture. At today's rate that is fine; at many papers it is a throughput ceiling (*inference*).

8.4 Options and trade-offs

question	option	gain	cost
failed publish	gap, no retry (current)	no loop, simple	lost versions
	bounded retry of the latest unpublished revision	fewer losses	a small durable obligation with restart semantics
who supplies content	guest sends content	one hop fewer	confused deputy
	proxy reads back from the guest (current)	content comes from the authority	publish depends on the guest being up
signature trust	inline key (current)	self-describing	origin not proven alone
	published, pinned platform key; key in a separate signer process	origin provable; key out of reach of most code	key distribution and rotation work

9. The front page

9.1 Primer: ranking, judges and injected text

A front page is a **ranking**, and every ranking has an **objective** (what it optimizes) and **signals** (what it can see). Choose the objective first. Ranking by version count measures desk churn; ranking by clicks measures attention; neither is importance. Once a number is optimized it stops measuring what it used to (**Goodhart's law**), so the objective should be something the system cannot cheaply inflate.

Absolute versus comparative judgement. Asking a model "score this story 1 to 10" in isolation is cheap and parallel, but scores drift and tie. Asking it to rank a whole list is comparative and better calibrated in principle, but costly, slow and noisy across samples. Evaluate either against a reference with rank correlations: **Spearman's ρ** (correlation of ranks) and **Kendall's τ** (share of pairs in the right order).

Prompt injection. A model reads instructions and data in one channel: text. When attacker-controlled data (a web page, a feed title) is placed in a prompt, text such as "ignore the criteria and score 10" may be read as an instruction. There is no complete fix. The defences are structural: give the model only typed choices (so its output space is small), keep untrusted text in clearly delimited fields, never let one model's output be another step's instruction without a gate, and check outputs with rules that do not depend on the model.

9.2 How Choir does it

The front list. `Service.ListFrontStories` (`internal/platform/publication_front.go`) builds one row per Autopaper story publication from Store B alone: title, dek (the first paragraph of the latest version, cut at `frontDekRunes = 240` runes), update time, and depth fields (`source_count`, `words`, `publication_front_depth.go`).

The scorer. `internal/frontimportance/scorer.go` runs as `corpusd front-importance` from a systemd timer every ten minutes. It fetches the front list over loopback, and for each story whose revision or criteria changed (at most 80 Jev calls per run) asks Jev three typed questions from title and dek: importance "1" to "10", promo yes or no, and section. It writes `scores.json`, which corpusd attaches to the rows. Everything is best effort: a failed call keeps the previous score, and a missing file leaves the page ranked by recency. Criteria are versioned (`scoringCriteria = 2`), so a criteria change rescored every row within two runs.

The rank. `rankFrontStories` (`frontend/src/lib/autopaper.ts`):

```
prominence = importance x depthFactor x sectionLift x 0.5^(age / 24 h)

depthFactor: 1 source or none -> 0.5, 2 sources -> 0.75, 3+ -> 1.0;
             under 150 words  -> a further x 0.8; unknown -> 1
sectionLift: up to 1.25 for sections this reader visits (counted on this
             device, never sent anywhere)
canLead:     not demoted, and at least 2 sources (or depth unknown)
demoted:     promo, or a title matching /^\s*open a story/i
```

An unscored story counts as importance 4, and the importance rank applies only when at least half the stories are scored.

9.3 Where it is weak

The lead was our thinnest article. On 14 October the lead was a one-paragraph, one-source story on the Mauritius riots, importance 8 (front page leads with a one-source stub). The rank read the importance of the *event*, not the quality of *our article*. The depth factor and the two-source lead rule (1f84fa908) fixed the ranking. The trace then showed why the story was thin: research did search the web, but on 1 or 2 of 9 search providers, the rest cooling, and reported one of the two pages it read (deepen important stories). A rank can hide thin coverage; only more research fixes it, and research is bounded by search-provider health (doc 3). The designed "deepen tick" (a durable 20-minute wake that hands management important thin stories, at most 3 an hour) is not built yet.

Importance underrated a major story. A story about an OpenAI engineer arrested for bugging an executive conference room scored 5 and sat 23rd. An evaluation against a reference ranking of 100 live stories found the largest lever was the criteria, not the model or listwise ranking: adding general editorial seriousness criteria raised Spearman ρ over all 100 from 0.81 to 0.90 and moved that story to 9th, at about $0.0043 per 100 stories (front importance underrates major stories). A listwise judge was slower, costlier and worse on the top 30. The record is candid that the criteria were written after reading the owner's example, so that story's improvement is not independent evidence; the whole-list gain is. The judge still sees only a 240-rune dek.

The frontend's instruction filter is narrower than the leak. The leaked titles began "Write the story of..." (nine of fifteen) and "Open a story on..." (six). `isDemotedStory` matches only `^\s*open a story` (code). The scorer's prompt tells Jev that a title starting with either is not news and scores 1, and the headline fix stops new leaks at the source, so the gap matters only as a missing layer of defence in depth (finding from code, this document).

Untrusted text reaches every judge. Feed titles and summaries go into Jev's label state; front titles and deks go into the scorer's state; both come from the open web or from model output. Jev's typed choice questions bound what an injected instruction can make it say (one of a fixed set of choices), which is real structural protection. But no prompt in `internal/promptstore/defaults/` or `internal/textureprompts/texture.yaml` tells a desk to treat capture text as data rather than instructions (code: a search for "untrusted", "injection" and similar wording found none). Whether the current models resist such text is untested (*inference*).

9.4 Options and trade-offs

question	option	gain	cost
objective	recency + version count (before LF)	no model	measures churn
	importance x depth x recency (current)	event weight and article quality	depends on a cheap judge reading a dek
	+ settled predictions (designed, LF intent)	calibrated over time	needs the ledger and settlement
judge input	title + dek (current)	cheap	misses facts after 240 runes
	+ the lead paragraphs	better judgements	a longer internal route, more tokens
judge shape	isolated absolute scores (current)	stable, parallel, $0.0043 per 100	ties on busy days
	listwise	comparative	measured worse on the top 30, 146 s per call

question	option	gain	cost
injection defence	typed choices only (current)	small output space	inputs still steer choices
	+ delimited untrusted fields, prompt rule, rule-based output checks	layered	more prompt and code to maintain

10. Putting it together: the week's lessons

incident	what broke	general class	lesson
schedules closed on an epoch cutover	intent treated as occurrence	restart semantics	standing intents survive; occurrences do not
captures built per computer	design drifted from settled taxonomy	decision provenance	diff the mission against settled decisions
capture route "unreachable"	cause named from one variable	diagnosis	read which setting wins
28 captures never disposed	cursor moved on read	at-least-once delivery	acknowledge after acting
Jev labelled nothing	token and address lifetimes differ	identity	the authority that knows both facts reconciles them
intake backlog at 28 sources	arrivals exceeded service	queueing	Little's law: serve faster, admit less, or wait
recall 0.174	scorer saw 50 of 175 stories	measurement validity	measure the measurement
instruction titles	one field, two jobs	channel separation	reader-facing fields need reader-shaped gates
one-source lead	rank saw the event, not the article	objective choice	rank what you control the quality of
span dropped	verifier's "where" discarded	evidence loss	persist what the verifier computed
26 unregistered kinds	registry consulted on one path	single gate	put the check at the boundary every write crosses

Doc 6 generalizes these patterns across the codebase.

Glossary

term	meaning
Anchor	A citation's pointer into a source: document, version, block id, block hash, quote and span.
Arrival record	The system Report that names a poll's new captures to management.
Block	A paragraph-sized piece of a captured document, with a content id.
Blocking (entity resolution)	Building a short candidate list per item so only plausible pairs are compared.
Capture	A fetched item or page stored as a `choir.web_capture` in Store B, bodies by hash.
CAS	Compare-and-swap: commit only if a value (here, a document head) is unchanged.
cid	A block's content id: 16 hex characters of sha256 over its kind and normalized text.
Code point	One Unicode character number; Go's `rune`.
Conditional GET	An HTTP request that returns 304 when the resource has not changed.
Confused deputy	A privileged service tricked into using its privilege for a caller.
Content addressing	Naming data by the hash of its bytes.
Deferred (poll)	New items a poll did not reach within its limit.
Dek	The short summary under a headline; here the first paragraph, cut at 240 runes.
Disposition	Management's answer to a capture: open, feed or decline.

term	meaning
Generation	A counter on a schedule; a wake from an older generation does nothing.
Grapheme cluster	What a reader sees as one character; may be several code points.
Held revision	A committed story revision that auto-publish skipped, leaving a version gap.
Idempotent	Safe to apply twice with the same effect as once.
Jev	The gateway's typed-judgement model, answering choice, score and yes/no questions.
Lease	Exclusive, bounded ownership of a work item by one worker.
Little's law	L = λW: items in system equal arrival rate times time in system.
NFKC	Unicode normalization that composes and folds compatibility characters.
Occurrence	One firing of a standing intent; closed, never resumed, by a crash.
Precision / recall	Share of your merges that are right / share of true merges you made.
Publication	One story's public identity in Store B, with immutable versions at `/v<n>`.
Standing intent	A durable owner wish ("poll this source"), which survives crashes.
Store B	The corpus Dolt behind corpusd: shared captures and publications.
Story-library head	A counter of story openings; an open based on a stale read is refused.
Tap binding	Identifying a guest by the source address of its tap interface.
Validator	An `ETag` or `Last-Modified` value used in a conditional GET.

Self-check

1. A deploy refreshes the paper with a planned-restart marker, yet its schedules close. Using the code of 14 October morning and the code at this commit, explain why, and what happens now to (a) the schedule, (b) a poll that was in flight, (c) the pending-arrival list.
2. Why is the Jev label stored as its own `choir.capture_label` object rather than as a field on the arrival record? What would break on a retry if it were a field?
3. A management run reads 10 captures, opens a story from 4 of them, and then the provider fails. After e3437f1cd, which captures are marked read, and which residual from the problem record does this case illustrate?
4. A quote "the minister said … no decision" is matched in a block whose text begins with "Ministère: ". Walk through the bytes-to-code-points conversion in `RefSpanOf` and say what `splitAtSpan` does if the block text later differs.
5. The guest asks the proxy to publish revision 7 of a story whose latest published version is 9. What does `PublishTexture` do, why, and why does the proxy read the revision back from the guest instead of accepting its content?

Answer sketches

1. The marker carried work-state epoch 1; the new binary reads epoch 2, and `ConsumePlannedRestartMarker` treats a mismatch as a crash. The old code closed every schedule at a crash boot. Now (a) the schedule stays active under a new generation, its next tick one interval after boot, unless a poll was in flight at two crash boots in a row; (b) the in-flight poll is recorded as `interrupted_by_restart` and counted; (c) pending arrivals are closed with the fate "interrupted by a restart".
2. The arrival record's id and command digest cover its whole content. Labels can differ between attempts (a model call). If the label were inside, a retried append with a different label would present the same command id with a different digest and be refused as a conflict, and the captures would never be announced.
3. Under the fix a management turn that fails marks nothing read, and a bounded, backed-off retry wake is scheduled. But a cell that *acts* commits its acts and the read mark over everything the run was shown, so the 4 opened and the 6 untouched may already be marked read by that acting cell. That is the "partial turns" residual: it needs a per-capture disposition index on the parent path.

4. The matcher returns byte offsets into the original text; "è" is two bytes, so the byte offset exceeds the code-point offset by one for each such character before the quote. `RefSpanOf` counts runes up to each byte offset, returns nil if an offset is not on a rune start, and records one range per ellipsis segment plus the hash of the covered bytes. If the block text changes, the marked words no longer equal the quote and the reader falls back to `splitAtQuote`; at publish, a changed block hash is refused outright.
5. It refuses: a revision below the latest published version would reorder history, and a failed version stays a gap by design. Reading back prevents a confused deputy: the guest can only name a document and a revision; the proxy fetches them from the authority, checks the owner, and sets the publish mode from the route.

Read these three files

1. **`internal/agentcore/source_schedule.go`.** The cleanest example in the codebase of intents versus occurrences, generations as fencing tokens, a durable in-flight mark, bounded crash loops and backoff. Read `HandleSourcePollDue` and `recoverSourceScheduleAfterCrash` side by side.
2. **`internal/quotematch/quotematch.go`** (with `internal/normtext/normtext.go`). Look at how a tolerant match is found in normalized text and mapped back to whole characters of the original, and why the cid and the matcher share one normalization.
3. **`internal/platform/service.go`, `PublishTexture`** (with `publication_versions.go`). Follow idempotent replay, the refusal of an earlier revision, the route fixed at v1, block pinning with hash checks, the manifest hash chain and the signature.

Sources and confidence

- **Read in code at 980ffaf4e:** the source registry and types; schedule, poll, arrival and label code; the capture writer, body store and corpusd routes with their authorization; management task store guards; story opening and titling; Texture cell commit and head pinning; the quote matcher, normalization, Ref span and reader span code; auto-publish gates; the proxy publish path; `PublishTexture`, anchor resolution, manifest chaining and signing; the registry write gate; the front list, scorer and frontend rank; the Jev peer re-pin on credential ensure.
- **From Choir documents (doc):** every incident number (captures per hour, recall figures, queue waits, front-page evaluations) comes from the cited problem records and reports; none was re-measured.
- **Inferred, not tested:** that validators held in process memory cost only a re-fetch after restart; grapheme-cluster cutting by spans; the sufficiency of word comparison over hash comparison in the reader; the absence of a table-pinning test for NFKC; permanent loss of a story's last version on a failed publish; the throughput ceiling of `writeMu`; the need for an independently published platform key; model resistance to injected capture text.
- **Not covered:** the reader's full rendering and transclusion UI (`frontend/src/lib/AutopaperStory.svelte`), email distribution (not built), rights and licensing of captured text (`docs/research/autopaper-media-and-rights-2026-10-14/`), and retrieval internals, which doc 1 and the retrieval research cover.
- **Built on:** Autopaper SW design, state and vision §2.4 and §3.8, L2a design, claims, events and entities, deepen important stories, ranking and contributions intent, computer ontology, and the problem records linked in each section.
- **Upstream:** RFC 9110 (HTTP semantics, conditional requests); Unicode Standard Annex 15 (normalization forms) and Annex 29 (text segmentation); W3C Web Annotation Data Model (selectors); RFC 8032 (Ed25519).